

LAPORAN PRATIKUM 2
PEMOGRAMAN BEROENTAS OBJEK

Disusun Oleh:

Rifqi Aditya

2511533002

Dosen Pengampu :

Nurfiah, S.ST, M.Kom

Asisten Praktikum :

Siti Fadhilah Rahmi



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum mata kuliah Pemograman Beroentasi Objek dengan judul “Mini Bank”.

Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pekan 2 sekaligus sebagai sarana pembelajaran yang membahas langkah langkah yang ada dalam praktikum serta penyelesaian tugas yang berupa khususnya terkait implementasi konsep enkapsulasi, koleksi objek, dan interaksi antar class dalam bahasa Java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca, khususnya dalam memperdalam pemahaman tentang pemrograman dasar dengan Java.

Padang, 19 September 2026

Rifqi Aditya

Daftar Isi

BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan	4
1.3 Manfaat.....	4
BAB II PEMBAHASAN.....	5
1.Langkah Kerja Praktik.....	5
1.1 Class Rekening	5
1.2 Class Main	8
1.3 Class Transaksi	12
2.Tugas	13
2.1 Integrasi Rekam Jejak Penarikan	13
2.2 Fitur Cetak Mutasi Rekening.....	14
3.Output	15
BAB III KESIMPULAN	17

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah mendorong hampir seluruh sektor kehidupan, termasuk sektor perbankan, untuk beralih ke sistem digital dalam mengelola data transaksi nasabah. Pencatatan transaksi yang akurat dan mudah ditelusuri menjadi kebutuhan mendasar dalam sistem perbankan, baik yang berskala besar maupun sistem sederhana yang dibangun untuk kepentingan pembelajaran. Dalam konteks pemrograman berorientasi objek, kebutuhan ini dapat diimplementasikan melalui pemodelan class yang saling berelasi, di mana setiap objek transaksi disimpan dan dikelola dalam sebuah koleksi data pada objek rekening pemiliknya.

Pada tugas ini, dikembangkan sebuah program sistem perbankan mini menggunakan bahasa Java yang terdiri dari tiga class utama, yaitu Rekening, Transaksi, dan Main. Program awal yang diberikan telah memiliki fitur dasar seperti pembukaan rekening, setor tunai, dan tarik tunai, namun fitur pencatatan riwayat transaksi belum terintegrasi sepenuhnya, khususnya pada method tarikTunai() dan belum tersedianya fitur untuk menampilkan seluruh riwayat mutasi rekening. Oleh karena itu, diperlukan pengembangan lebih lanjut untuk melengkapi fitur tersebut agar sistem dapat mencatat dan menampilkan riwayat transaksi secara utuh, baik transaksi kredit maupun debit.

1.2 Tujuan

1. Mengintegrasikan fitur pencatatan riwayat transaksi pada method tarikTunai() agar setiap penarikan yang berhasil tercatat sebagai objek Transaksi dengan jenis "Debit".
2. Membangun method cetakMutasi() pada class Rekening untuk menampilkan seluruh riwayat transaksi yang tersimpan menggunakan struktur perulangan.
3. Mengimplementasikan menu baru pada class Main agar pengguna dapat mengakses fitur cetak mutasi rekening secara langsung melalui antarmuka program.

1.3 Manfaat

1. Melatih pemahaman praktis mahasiswa mengenai cara merepresentasikan entitas dunia nyata ke dalam struktur kode program berorientasi objek.
2. Mengasah logika pemrograman dalam menangani atau validasi input agar program tidak mengalami errorsaat menerima data yang tidak valid.
3. Memberikan gambaran awal mengenai pengelolaan data dan teknik pencarian elemen pada Collection menggunakan perulangan dan perbandingan.

BAB II

PEMBAHASAN

1.Langkah Kerja Praktik

1.1 Class Rekening

Pada class ini mendefinisikan atribut dan method dari objek rekening,

1.1.1 Kode Program

```
1 package Pekan2_2511533002;
2
3 import java.util.ArrayList;
4
5 public class Rekening {
6     String nomorRekening;
7     String namaPemilik;
8     double saldo;
9
10    ArrayList<Transaksi> riwayatTransaksi;
11
12    public Rekening(String nomor, String nama, double saldoAwal) {
13        nomorRekening = nomor;
14        namaPemilik = nama;
15        saldo = saldoAwal;
16
17        this.riwayatTransaksi = new ArrayList<>();
18
19        System.out.println("Rekening atas nama " + namaPemilik + "Berhasil dibuat dengan saldo Rp" + saldo);
20    }
21
22    public void setorTunai(double nominal) {
23        if (nominal > 0) {
24            saldo += nominal;
25
26            String idTrx = "TRX-S-" + System.currentTimeMillis();
27            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
28            riwayatTransaksi.add(trxBaru);
29
30            System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini Rp" + saldo);
31        } else {
32            System.out.println("Gagal : Nominal setor harus lebih dari 0!");
33        }
34    }
35
36    public void tarikTunai(double nominal) {
37        if (nominal < 10000) {
38            System.out.println("Transaksi Gagal: Nominal minimal adalah Rp. 10.000");
39        } else if (nominal > saldo) {
40            System.out.println("Transaksi Gagal: Nominal penarikan tidak mencukupi saldo Rp. " + saldo);
41        } else {
42            saldo -= nominal;
43
44            String idTrx = "TRX-T-" + System.currentTimeMillis();
45            Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
46            riwayatTransaksi.add(trxBaru);
47
48            System.out.println("Sukses: Anda menarik saldo Rp" + nominal + ". Sisa saldo = Rp" + saldo);
49        }
50    }
51
52    public void cetakMutasi() {
53        System.out.println("---- MUTASI REKENING " + nomorRekening + " ----");
54
55        if (riwayatTransaksi.isEmpty()) {
56            System.out.println("Belum ada transaksi pada rekening ini");
57        } else {
58            for (Transaksi trx : riwayatTransaksi) {
59                trx.cetakDetail();
60            }
61        }
62
63        System.out.println("Saldo Akhir : Rp" + saldo);
64        System.out.println("-----");
65    }
66
67    public void cekInformasi() {
68        System.out.println("---- INFO REKENING ----");
69        System.out.println("No. Rekening : " + nomorRekening);
70        System.out.println("Nama Pemilik : " + namaPemilik);
71        System.out.println("Saldo Akhir : Rp" + saldo);
72        System.out.println("-----");
73    }
74 }
```

1.1.2 Uraian Kode Program

1.

```
String nomorRekening;  
String namaPemilik;  
double saldo;
```

 - Pada bagian ini disebut constructor, constructor ini berfungsi menginisialisasi ketiga atribut dengan nilai yang dikirim ke Main, lalu mencetak konfirmasi ke output.
2.

```
public void setorTunai(double nominal) {  
    if (nominal > 0) {  
        saldo += nominal;  
  
        String idTrx = "TRX-S-" + System.currentTimeMillis();  
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);  
        riwayatTransaksi.add(trxBaru);  
  
        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini Rp" + saldo);  
    } else {  
        System.out.println("Gagal : Nominal setor harus lebih dari 0!");  
    }  
}
```

 - Untuk menambah saldo rekening. Validasi terlebih dahulu apakah $\text{nominal} > 0$. Jika valid, nilai saldo bertambah sebesar nominal, program membuat ID transaksi unik berbasis waktu milidetik dengan awalan "TRX-S-", membuat objek transaksi baru bertipe "Kredit" sebesar nominal, dan mencatatnya ke dalam riwayatTransaksi. Setelah itu, program mencetak pesan konfirmasi setor tunai berhasil beserta saldo saat ini. Jika syarat tersebut tidak terpenuhi ($\text{nominal} \leq 0$), setoran dibatalkan dengan mencetak pesan penolakan "Gagal : Nominal setor harus lebih dari 0!"
3.

```
public void tarikTunai(double nominal) {  
    if (nominal < 10000) {  
        System.out.println("Transaksi Gagal: Nominal minimal adalah Rp. 10.000");  
    } else if (nominal > saldo) {  
        System.out.println("Transaksi Gagal: Nominal penarikan tidak mencukupi saldo Rp. " + saldo);  
    } else {  
        saldo -= nominal;  
  
        String idTrx = "TRX-T-" + System.currentTimeMillis();  
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);  
        riwayatTransaksi.add(trxBaru);  
  
        System.out.println("Sukses: Anda menarik saldo Rp" + nominal + ". Sisa saldo = Rp" + saldo);  
    }  
}
```

 - berfungsi untuk memproses penarikan uang dari rekening. Method ini memvalidasi terlebih dahulu apakah $\text{nominal} < 10.000$, akan mencetak pesan gagal karena nominal kurang dari batas minimal atau $\text{nominal} > \text{saldo}$, akan mencetak pesan gagal karena saldo tidak mencukupi. Jika salah satu syarat kegagalan tersebut terpenuhi, penarikan dibatalkan dengan pesan penolakan yang sesuai. Jika semua syarat validasi terpenuhi, transaksi berhasil dan nilai saldo dikurangi sebesar nominal yang ditarik.
4.

```
String idTrx = "TRX-T-" + System.currentTimeMillis();  
Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);  
riwayatTransaksi.add(trxBaru);  
  
System.out.println("Sukses: Anda menarik saldo Rp" + nominal + ". Sisa saldo = Rp" + saldo);
```

 - Untuk mencatat dan menampilkan riwayat penarikan saldo yang berhasil. Program membuat ID transaksi unik berbasis waktu milidetik dengan awalan "TRX-T-", lalu membuat objek transaksi baru bertipe "Debit" sebesar nominal, dan mencatatnya ke dalam

riwayatTransaksi. Terakhir, program mencetak pesan konfirmasi sukses beserta nominal penarikan dan sisa saldo terbaru.

```
public void cekInformasi() {  
    System.out.println("--- INFO REKENING ---");  
    System.out.println("No. Rekening : " + nomorRekening);  
    System.out.println("Nama Pemilik : " + namaPemilik);  
    System.out.println("Saldo Akhir : Rp" + saldo);  
    System.out.println("-----");  
}
```

5. ➤ hanya menampilkan data objek: nomor rekening, nama pemilik, dan saldo akhir, ke output.

```
public void cetakMutasi() {  
    System.out.println("--- MUTASI REKENING " + nomorRekening + " ---");  
  
    if (riwayatTransaksi.isEmpty()) {  
        System.out.println("Belum ada transaksi pada rekening ini");  
    } else {  
        for (Transaksi trx : riwayatTransaksi) {  
            trx.cetakDetail();  
        }  
    }  
  
    System.out.println("Saldo Akhir : Rp" + saldo);  
    System.out.println("-----");  
}
```

6. ➤ Untuk menampilkan mutasi atau riwayat transaksi rekening. Method cetakMutasi terlebih dahulu mencetak header mutasi rekening sesuai nomorRekening. Selanjutnya, method melakukan pengecekan apakah riwayatTransaksi.isEmpty() belum ada riwayat transaksi. Jika belum ada transaksi, program menampilkan pesan pemberitahuan bahwa belum ada transaksi. Jika terdapat riwayat transaksi, program melakukan perulangan untuk mencetak detail tiap transaksi trx.cetakDetail(). Di akhir, program mencetak saldo akhir beserta garis penutup.

1.2 Class Main

Pada main berisi method untuk menyiapkan Scanner untuk membaca input pengguna, sebuah variabel untuk objek, lalu menjalankan perulangan while yang menampilkan menu dan memproses pilihan pengguna melalui struktur switch-case.

1.2.1 Kode Program

```
1 package Pekani_2511533002;
2 import java.util.Scanner;
3 import java.util.ArrayList;
4 public class Main {
5     public static void main(String[] args) {
6         Scanner input = new Scanner(System.in);
7         boolean isRunning = true;
8         ArrayList<Rekening> daftarRekening = new ArrayList<>();
9         Rekening akunAktif = null;
10
11         System.out.println("=== SISTEM PERBANKAN MINI ===");
12
13         while (isRunning) {
14             System.out.println("\nMenu Utama : ");
15             System.out.println("1. Buka rekening baru");
16             System.out.println("2. Setor tunai");
17             System.out.println("3. Tarik tunai");
18             System.out.println("4. Cek informasi rekening");
19             System.out.println("5. Ganti akun");
20             System.out.println("0. Keluar");
21             System.out.println("Pilih menu : ");
22
23             int Pilihan = input.nextInt();
24             input.nextLine(); //membersihkan buffer enter
25
26             switch (Pilihan) {
27                 case 1:
28                     System.out.print("Masukkan No Rekening : ");
29                     String no = input.nextLine();
30                     System.out.print("Masukkan Nama Pemilik : ");
31                     String nama = input.nextLine();
32                     System.out.print("Masukkan Saldo Awal : ");
33                     double saldo = input.nextDouble();
34
35                     //instansiasi objek / menjalankan constructor
36                     akunAktif = new Rekening(no, nama, saldo);
37                     daftarRekening.add(akunAktif);
38                     break;
39
40                 case 2:
41                     if (akunAktif == null) {
42                         System.out.println("Error : Mohon maaf, anda belum memiliki nomor rekening!");
43                     } else {
44                         System.out.print("Masukkan nominal setor : ");
45                         double setor = input.nextDouble();
46                         akunAktif.setorTunai(setor); //Memanggil behavior / method
```

```
1 package Pekani_2511533002;
2 import java.util.Scanner;
3 import java.util.ArrayList;
4 public class Main {
5     public static void main(String[] args) {
6         Scanner input = new Scanner(System.in);
7         boolean isRunning = true;
8         ArrayList<Rekening> daftarRekening = new ArrayList<>();
9         Rekening akunAktif = null;
10
11         System.out.println("=== SISTEM PERBANKAN MINI ===");
12
13         while (isRunning) {
14             System.out.println("\nMenu Utama : ");
15             System.out.println("1. Buka rekening baru");
16             System.out.println("2. Setor tunai");
17             System.out.println("3. Tarik tunai");
18             System.out.println("4. Cek informasi rekening");
19             System.out.println("5. Ganti akun");
20             System.out.println("0. Keluar");
21             System.out.println("Pilih menu : ");
22
23             int Pilihan = input.nextInt();
24             input.nextLine(); //membersihkan buffer enter
25
26             switch (Pilihan) {
27                 case 1:
28                     System.out.print("Masukkan No Rekening : ");
29                     String no = input.nextLine();
30                     System.out.print("Masukkan Nama Pemilik : ");
31                     String nama = input.nextLine();
32                     System.out.print("Masukkan Saldo Awal : ");
33                     double saldo = input.nextDouble();
34
35                     //instansiasi objek / menjalankan constructor
36                     akunAktif = new Rekening(no, nama, saldo);
37                     daftarRekening.add(akunAktif);
38                     break;
39
40                 case 2:
41                     if (akunAktif == null) {
42                         System.out.println("Error : Mohon maaf, anda belum memiliki nomor rekening!");
43                     } else {
44                         System.out.print("Masukkan nominal setor : ");
45                         double setor = input.nextDouble();
46                         akunAktif.setorTunai(setor); //Memanggil behavior / method
```

```

50     case 3:
51         if (akunAktif == null) {
52             System.out.println("Error : Mohon maaf, anda belum memiliki nomor rekening!");
53         } else {
54             System.out.print("Masukkan nominal tarik : ");
55             double tarik = input.nextDouble();
56             akunAktif.tarikTunai(tarik);
57         }
58         break;
59
60     case 4:
61         if (akunAktif == null) {
62             System.out.println("Error : Anda belum membuka rekening!");
63         } else {
64             akunAktif.cekInformasi();
65         }
66         break;
67
68     case 5:
69         if (daftarRekening.isEmpty()) {
70             System.out.println("Error : Belum ada rekening yang terdaftar!");
71         } else {
72             System.out.print("Masukkan No Rekening yang ingin diaktifkan : ");
73             String cariNo = input.nextLine();
74             Rekening ditemukan = null;
75             for (Rekening r : daftarRekening) {
76                 if (r.nomorRekening.equals(cariNo)) {
77                     ditemukan = r;
78                     break;
79                 }
80             }
81             if (ditemukan == null) {
82                 System.out.println("Error : No Rekening " + cariNo + " tidak ditemukan!");
83             } else {
84                 akunAktif = ditemukan;
85                 System.out.println("Berhasil! Akun aktif sekarang: " + akunAktif.namaPemilik + " (" + akunAktif.nomorRekening + ")");
86             }
87         }
88         break;
89
90     case 6:
91         if (akunAktif == null) {
92             System.out.println("Error : Anda belum membuka rekening!");
93         } else {
94             akunAktif.cetakMutasi();
95         }
96         break;
97
98     case 0:
99         isRunning = false;

```

```

100         System.out.println("Sistem ditutup. Terima Kasih!");
101         break;
102
103         default:
104             System.out.println("Pilihan tidak valid!");
105     }
106 }
107 input.close();
108 }
109 }

```

1.2.2 Uraian Kode Program

```

switch (Pilihan) {
case 1:
    System.out.print("Masukkan No Rekening : ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik : ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal : ");
    double saldo = input.nextDouble();

    //Instansiasi objek / menjalankan constructor
    akunAktif = new Rekening(no, nama, saldo);
    daftarRekening.add(akunAktif);
    break;

```

1.

- Membaca no rekening, nama, dan saldo awal dari input, lalu membuat objek baru dengan (no, nama, saldo) dan menyimpannya ke akunAktif.

```

case 2:
    if (akunAktif == null) {
        System.out.println("Error : Mohon maaf, anda belum memiliki nomor rekening!");
    }else {
        System.out.print("Masukkan nominal setor : ");
        double setor = input.nextDouble();
        akunAktif.setorTunai(setor); //Memanggil behavior / method
    }
    break;

```

2. ➤ Mengecek apakah `akunAktif == null` sebelum meminta nominal dan memanggil `akunAktif`, sehingga program tidak error bila belum ada rekening.

```

case 3:
    if (akunAktif == null) {
        System.out.println("Error : Mohon maaf, anda belum memiliki nomor rekening!");
    }else {
        System.out.print("Masukkan nominal tarik : ");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);
    }
    break;

```

3. ➤ Mengecek terlebih dahulu apakah `akunAktif == null` untuk menampilkan pesan error. Jika akun aktif ada, program meminta input nominal penarikan via `input.nextDouble()` lalu memanggil method `akunAktif.tarikTunai` untuk memproses penarikan saldo.

```

case 4:
    if (akunAktif == null) {
        System.out.println("Error : Anda belum membuka rekening!");
    }else {
        akunAktif.cekInformasi();
    }
    break;

```

4. ➤ Sama seperti case 2, mengecek `akunAktif == null` lebih dulu, baru memanggil `akunAktif.cekInformasi()` untuk menampilkan data rekening yang sedang aktif.

```

case 5:
    if (daftarRekening.isEmpty()) {
        System.out.println("Error : Belum ada rekening yang terdaftar!");
    }else {
        System.out.print("Masukkan No Rekening yang ingin diaktifkan : ");
        String cariNo = input.nextLine();
        Rekening ditemukan = null;
        for (Rekening r : daftarRekening) {
            if (r.nomorRekening.equals(cariNo)) {
                ditemukan = r;
                break;
            }
        }
        if (ditemukan == null) {
            System.out.println("Error : No Rekening " + cariNo + " tidak ditemukan!");
        }else {
            akunAktif = ditemukan;
            System.out.println("Berhasil! Akun aktif sekarang: " + akunAktif.namaPemilik + " (" + akunAktif.nomorRekening + ")");
        }
    }
    break;

```

5. ➤ Memeriksa apakah `daftarRekening.isEmpty()` untuk mencegah pencarian jika belum ada rekening yang dibuat. Jika ada, program meminta input nomor rekening dan menelusuri `daftarRekening` menggunakan perulangan serta perbandingan. Jika nomor rekening ditemukan, maka `akunAktif` diubah ke objek tersebut dan menampilkan konfirmasi sukses, jika tidak ditemukan, program menampilkan pesan error.

```

90         case 6:
91             if (akunAktif == null) {
92                 System.out.println("Error : Anda belum membuka rekening!");
93             } else {
94                 akunAktif.cetakMutasi();
95             }
96             break;
97

```

- 6.
- validasi terlebih dahulu apakah `akunAktif == null` atau pengguna belum memilih atau membuka rekening. Jika syarat tersebut terpenuhi, proses dibatalkan dengan mencetak pesan penolakan "Error : Anda belum membuka rekening!". Jika valid atau terdapat akun aktif, program memanggil method `akunAktif.cetakMutasi()` untuk menampilkan mutasi transaksi rekening tersebut, lalu menghentikan eksekusi blok switch menggunakan perintah `break`.

```

case 0:
    isRunning = false;
    System.out.println("Sistem ditutup. Terima Kasih!");
    break;

default:
    System.out.println("Pilihan tidak valid!");

```

- 7.
- Mengubah `isRunning` menjadi `false` sehingga perulangan `while` berhenti, lalu mencetak pesan penutup.

1.3 Class Transaksi

1.3.1 Kode Program

```
1 package Pekan2_2511533002;
2
3 public class Transaksi {
4     String idTransaksi;
5     String jenis;
6     double nominal;
7
8     //Constructor
9     public Transaksi(String id, String jenis, Double nominal) {
10         this.idTransaksi = id;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
14
15     public void cetakDetail() {
16         System.out.println("ID : " + idTransaksi + " | Jenis : " + jenis + " | Nominal : Rp" + nominal);
17     }
18 }
```

1.3.2 Uraian Kode Program

```
3 public class Transaksi {
4     String idTransaksi;
5     String jenis;
6     double nominal;
```

1.

- Pada bagian ini disebut atribut instansi, variabel ini berfungsi untuk menyimpan informasi utama dari sebuah transaksi, yaitu ID transaksi, jenis transaksi, dan nominal uang.

```
    public Transaksi(String id, String jenis, Double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }
```

2.

- Pada bagian ini disebut constructor, constructor ini berfungsi menginisialisasi ketiga atribut yaitu idTransaksi, jenis, dan nominal dengan nilai yang dikirim dari parameter saat pembuatan objek transaksi baru.

```
    public void cetakDetail() {
        System.out.println("ID : " + idTransaksi + " | Jenis : " + jenis + " | Nominal : Rp" + nominal);
    }
```

3.

- Pada bagian ini disebut method, method ini berfungsi untuk menampilkan seluruh rincian informasi transaksi seperti ID transaksi, jenis transaksi, dan nominal ke output layar secara terstruktur.

2. Tugas

2.1 Integrasi Rekam Jejak Penarikan

2.1.1 Kode Program

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal: Nominal minimal adalah Rp. 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Nominal penarikan tidak mencukupi saldo Rp. " + saldo);
    } else {
        saldo -= nominal;

        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Sukses: Anda menarik saldo Rp" + nominal + ". Sisa saldo = Rp" + saldo);
    }
}
```

2.1.2 Uraian Kode Program

1.

```
if (nominal < 10000) {
    System.out.println("Transaksi Gagal: Nominal minimal adalah Rp. 10.000");
}
```

 - Menolak transaksi jika nominal di bawah batas minimal Rp10.000,
2.

```
else if (nominal > saldo) {
    System.out.println("Transaksi Gagal: Nominal penarikan tidak mencukupi saldo Rp. " + saldo);
}
```

 - Menolak transaksi jika nominal yang diminta melebihi saldo yang tersedia, sehingga saldo tidak mungkin menjadi minus.
3.

```
} else {
    saldo -= nominal;
}
```

 - Mengurangi nilai saldo sebesar nominal penarikan yang dimasukkan jika seluruh kondisi validasi sebelumnya tidak ada yang terpenuhi.
4.

```
String idTrx = "TRX-T-" + System.currentTimeMillis();
Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
riwayatTransaksi.add(trxBaru);
```

 - Membuat ID transaksi unik berbasis waktu milidetik dengan awalan "TRX-T-", lalu membuat objek transaksi baru bertipe "Debit" sebesar nominal, dan menyimpannya ke dalam daftar riwayatTransaksi untuk mencatat rekam jejak transaksi yang berhasil.

2.2 Fitur Cetak Mutasi Rekening

2.2.1 Kode Program

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error : Anda belum membuka rekening!");
    } else {
        akunAktif.cetakMutasi();
    }
    break;
```

```
public void cetakMutasi() {
    System.out.println("--- MUTASI REKENING " + nomorRekening + " ---");

    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi trx : riwayatTransaksi) {
            trx.cetakDetail();
        }
    }

    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}
```

2.2.2 Uraian Kode Program

1.

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error : Anda belum membuka rekening!");
```

 - Menolak permintaan pada case 6 jika akunAktif bernilai null pengguna belum memilih atau membuka rekening.
2.

```
    } else {
        akunAktif.cetakMutasi();
```

 - Memanggil method cetakMutasi() pada objek akunAktif untuk menampilkan riwayat mutasi rekening jika syarat pengecekan sebelumnya tidak terpenuhi atau akun terverifikasi aktif.
3.

```
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
```

 - Memeriksa apakah daftar riwayatTransaksi kosong menggunakan method isEmpty(), jika kosong maka program mencetak pesan pemberitahuan "Belum ada transaksi pada rekening ini".
4.

```
    } else {
        for (Transaksi trx : riwayatTransaksi) {
            trx.cetakDetail();
        }
```

 - Mengeksekusi blok kode jika daftar riwayat transaksi tidak kosong, dengan melakukan perulangan untuk memanggil method cetakDetail() pada setiap objek transaksi trx di dalam koleksi riwayatTransaksi guna menampilkan rincian riwayat transaksi satu per satu.

3.Output

1. Melakukan Setor Tunai

```
=== SISTEM PERBANKAN MINI ===

Menu Utama :
1. Buka rekening baru
2. Setor tunai
3. Tarik tunai
4. Cek informasi rekening
5. Ganti akun
6. Cek Mutasi
0. Keluar
Pilih menu :
1
Masukkan No Rekening : 2511533002
Masukkan Nama Pemilik : Rifqi
Masukkan Saldo Awal : 50000
Rekening atas nama RifqiBerhasil dibuat dengan saldo Rp50000.0

Menu Utama :
1. Buka rekening baru
2. Setor tunai
3. Tarik tunai
4. Cek informasi rekening
5. Ganti akun
6. Cek Mutasi
0. Keluar
Pilih menu :
2
Masukkan nominal setor : 20000
Setor tunai Rp20000.0 berhasil. Saldo saat ini Rp70000.0
```

2. Melakukan Tarik Tunai

```
Menu Utama :
1. Buka rekening baru
2. Setor tunai
3. Tarik tunai
4. Cek informasi rekening
5. Ganti akun
6. Cek Mutasi
0. Keluar
Pilih menu :
3
Masukkan nominal tarik : 30000
Sukses: Anda menarik saldo Rp30000.0. Sisa saldo = Rp40000.0
```

3. Melakukan Pengecekan Mutasi

```
Menu Utama :
1. Buka rekening baru
2. Setor tunai
3. Tarik tunai
4. Cek informasi rekening
5. Ganti akun
6. Cek Mutasi
0. Keluar
Pilih menu :
6
--- MUTASI REKENING 2511533002 ---
ID : TRX-S-1789828612632 | Jenis : Kredit | Nominal : Rp20000.0
ID : TRX-T-1789828632196 | Jenis : Debit | Nominal : Rp30000.0
Saldo Akhir : Rp40000.0
-----
```

4. Melakukan Pengecekan Mutasi Tanpa Transaksi

```
=== SISTEM PERBANKAN MINI ===

Menu Utama :
1. Buka rekening baru
2. Setor tunai
3. Tarik tunai
4. Cek informasi rekening
5. Ganti akun
6. Cek Mutasi
0. Keluar
Pilih menu :
6
Gagal : Anda belum membuka rekening!
```

BAB III

KESIMPULAN

Berdasarkan hasil pengembangan dan pengujian program, dapat disimpulkan bahwa integrasi fitur pencatatan riwayat transaksi berhasil diterapkan dengan baik pada sistem perbankan mini ini. Penambahan logika pencatatan objek Transaksi pada method tarikTunai() memastikan bahwa hanya transaksi penarikan yang benar-benar berhasil yang akan tercatat ke dalam riwayat, sehingga data yang tersimpan tetap akurat dan konsisten dengan kondisi saldo. Selain itu, method cetakMutasi() yang dibangun pada class Rekening terbukti mampu menampilkan seluruh riwayat transaksi secara berurutan menggunakan struktur perulangan for-each, lengkap dengan penanganan kondisi ketika belum ada transaksi yang tercatat.

Secara keseluruhan, hasil pengujian menunjukkan bahwa seluruh fitur berjalan sesuai dengan ketentuan yang ditetapkan, mulai dari validasi nominal penarikan, pencatatan jenis transaksi, hingga penambahan menu baru pada Main.java yang memungkinkan pengguna mengakses fitur mutasi rekening dengan mudah. Pengerjaan tugas ini juga memberikan pemahaman yang lebih mendalam mengenai pentingnya struktur data koleksi dan interaksi antar objek dalam merancang sebuah sistem yang terintegrasi, terstruktur, dan mudah dikembangkan lebih lanjut.